

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Higher Institute of Sciences



Projet de fin d'étude

En : Mathématiques et Informatique

Spécialité : Ingénierie de Systèmes d'Information & Logiciels

Intitulé :

Vectorization Automatique d'Images Matricielles par Deep Learning

Encadré par :

ABDELLAHOUM Hamza

Présenté par :

BITAM Amir

AITHAMOUDI Zakaria

Membres du Jury :

????

????

Soutenu le :

??/07/2026

Année universitaire : 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude envers Allah, le Tout-Puissant, pour nous avoir accordé la force, la patience et la persévérance nécessaires pour mener à bien ce projet de fin d'étude.

Nous remercions sincèrement notre encadreur **ABDELLAHOU** **Hamza** pour sa précieuse guidance, son soutien constant et ses précieux conseils tout au long de ce projet.

Nos vifs remerciements s'adressent également aux membres du jury **????** et **????** pour avoir accepté d'évaluer notre travail.

Dédicace

Résumé

Nous proposons dans le cadre de notre projet de fin d'études un modèle d'IA qui traite le zoom et le dézoom dans les images numériques. En zoomant sur une image de taille donnée, sa qualité est souvent altérée et l'image peut devenir pixélisée. Notre travail consiste à améliorer ces images en augmentant leur qualité.

En ce qui concerne le dézoom, en réduisant les dimensions de l'image, il faut minimiser le nombre de pixels tout en préservant l'esthétique de l'image.

Nous avons créé un modèle CNN avec une architecture adaptée, piloté par des métaheuristiques (algorithme génétique) permettant de sélectionner les paramètres optimaux afin d'obtenir de bonnes performances avec un nombre minimal de paramètres.

Mots-clés : Redimensionnement d'images, Super Résolution, Réseaux de Neurones Convolutifs (CNN), Algorithme Génétique, Métaheuristiques, PSNR, SSIM.

Abstract

Keywords : Image Resizing, Super Resolution, Convolutional Neural Networks (CNN), Genetic Algorithm, Metaheuristics, PSNR, SSIM.

Liste des Acronymes

XML	eXtensible Markup Language
SVG	Scalable Vector Graphics
GA	Genetic Algorithm
CNN	Convolutional Neural Networks
PSNR	Peak Signal to Noise Ratio
HR	High Resolution
LR	Low Resolution
SSIM	Structural SIMilarity
SRCNN	Super Resolution Convolutional Neural Network
SRGAN	Super Resolution Generative Adversarial Network
MAE	Mean Absolute Error
MSE	Mean Squared Error
ReLU	Rectified Linear Unit
ResNet	Residual Network
SE	Squeeze-and-Excitation
CAR	Content Adaptive Resampler
FARF	Feature Augmented Random Forest
DRDN	Deep Residual Dense Network
EDSAN	Enhanced Dense Space Attention Network
PPI	Pixels Per Inch
ViT	Vision Transformer
UDF	Unsigned Distance guided Focal loss

Table des matières

Remerciements	i
Dédicace	ii
Résumé	iii
Abstract	iv
Liste des Acronymes	v
Introduction Générale	2
1 État de l’Art	3
1.1 Introduction	3
1.2 Définitions liées à la vectorisation d’images	3
1.2.1 Images matricielles	3
1.2.1.1 Pixels	4
1.2.2 Images vectorielles	4
1.2.2.1 Courbes de Bézier	5
1.2.2.2 Format SVG	5
1.3 Apprentissage automatique et apprentissage profond	5
1.3.1 Apprentissage automatique	5
1.3.1.1 Apprentissage supervisé	6
1.3.1.2 Apprentissage non supervisé	6
1.3.1.3 Apprentissage auto-supervisé	6
1.3.2 Apprentissage profond	7
1.3.2.1 Réseaux de neurones	7
1.3.2.2 Réseaux convolutionnels	7
1.3.2.3 Transformers	8
1.3.2.4 Vision Transformers	8
1.4 Mécanismes d’attention	8
1.4.1 Attention	9
1.4.2 Self-attention	9
1.4.3 Cross-attention	9
1.5 Concepts utilisés dans la vectorisation par apprentissage profond	10
1.5.1 Superpixels	10
1.5.2 Rendu différentiable	11

1.5.3	Fonctions de perte	11
1.5.4	Apprentissage coarse-to-fine	11
1.6	Évaluation des résultats	12
1.6.1	MSE	12
1.6.2	PSNR	13
1.6.3	SSIM	13
1.6.4	LPIPS	14
1.6.5	Temps d'inférence	14
1.7	Travaux connexes	15
1.7.1	Approches algorithmiques	15
1.7.1.1	Potrace	15
1.7.2	Approches basées sur l'optimisation	15
1.7.2.1	DiffVG	15
1.7.2.2	LIVE	15
1.7.3	Approches basées sur l'apprentissage profond	15
1.7.3.1	Im2Vec	15
1.7.3.2	SuperSVG	15
1.8	Synthèse comparative des approches	15
1.9	Problématique	15
1.10	Conclusion	15
2	Approche Proposée	16
2.1	Introduction	16
2.2	Dataset et prétraitement	16
2.2.1	Le prétraitement des données	16
2.2.1.1	Former les paires d'images	16
2.2.1.2	Augmentation de données	16
2.3	Méthode proposée	16
2.3.1	Définir les hyperparamètres évalués par les métaheuristiques	16
2.3.1.1	L'Algorithme Génétique	16
2.3.1.2	Les paramètres à optimiser	16
2.3.1.3	Évaluation d'une solution	16
2.3.2	Architecture	16
2.3.2.1	Sur-échantillonnage (Upsampling)	16
2.3.2.2	Sous-échantillonnage (Downsampling)	16
2.4	Conclusion	16
3	Expérimentations et Résultats	17
3.1	Introduction	17
3.2	Environnement de développement	17

3.2.1	Hardware	17
3.2.2	Software	17
3.3	Dataset utilisé	17
3.4	Prétraitement	17
3.5	Évaluation	17
3.5.1	Évaluation quantitative	17
3.5.2	Fonction de perte	17
3.6	Paramétrage de l'Algorithme Génétique	17
3.7	Sous-échantillonnage	18
3.7.1	Hyperparamètres avec la métaheuristique (GA)	18
3.7.2	Résultats	18
3.7.2.1	Résultats quantitatifs	18
3.7.2.2	Résultats qualitatifs	18
3.8	Sur-échantillonnage	18
3.8.1	Hyperparamètres	18
3.8.2	Résultats	19
3.8.2.1	Résultats quantitatifs	19
3.8.2.2	Résultats qualitatifs	19
3.9	Conclusion	19
Conclusion Générale		20
Bibliographie		21

Table des figures

1.1	Comparaison entre une image matricielle (gauche) sujette à la pixellisation lors de l'agrandissement, et une image vectorielle (droite) conservant sa netteté grâce à sa définition mathématique	4
1.2	Exemple du contenu d'un fichier .svg	5

Liste des tableaux

3.1	Spécifications de la machine Kaggle	17
3.2	Valeurs du PSNR en fonction du nombre de générations	18
3.3	Résultats du downsampling (Set5) — PSNR / SSIM	18
3.4	Comparaison quantitative sur Set5 (PSNR en dB)	19

Introduction Générale

Le traitement d'images est un domaine de recherche important qui trouve des applications dans divers domaines tels que la médecine, la surveillance, la sécurité, la robotique, l'automobile, etc.

L'un des problèmes les plus courants dans ce domaine est la qualité de l'image qui peut être altérée lorsqu'elle est agrandie ou réduite en taille.

Ce mémoire est organisé comme suit :

- **Chapitre 1** : État de l'art — revue de littérature sur le redimensionnement d'images et les approches existantes.
- **Chapitre 2** : Approche proposée — architecture des modèles CNN et sélection des hyperparamètres par méta heuristiques.
- **Chapitre 3** : Expérimentations et Résultats — évaluation quantitative et qualitative avec comparaisons.

Chapitre 1

État de l'Art

1.1 Introduction

L'intelligence artificielle par le biais de l'apprentissage automatique a imposé un changement de paradigme dans de nombreux domaines[1], en particulier l'informatique et ses différentes disciplines. La vision par ordinateur est un parfait exemple de ce changement de paradigme, où la spécialité s'est dirigé de l'application d'algorithmes "naïf" à l'utilisation de réseaux de neurones pour le traitement et l'extraction d'informations de l'image, cette évolution dans les techniques utilisées par la vision par ordinateur est illustré dans l'évolution de la vectorisation d'images. Dans ce chapitre, nous exposerons d'abord les concepts clés à la vectorisation d'images, à l'apprentissage automatique, et les architectures utilisées tout au long de ce travail, ceci nous permettra de mieux appréhender ensuite les travaux effectués sur le sujet.

1.2 Définitions liées à la vectorisation d'images

Dans cette section nous tenterons de définir les termes et concepts lié à la transformation d'image matricielles en images vectorielles, dite, vectorisation d'images. En soulignant notamment la différence entre les images dites matricielles et vectorielles. En opposant ainsi les deux types d'images nous pourrions mieux cerner la motivation derrière ce travail et pourquoi la vectorisation d'image constitue un exercice complexe de la vision par ordinateur.

1.2.1 Images matricielles

Les images matricielles pourrait être décrites de la même manière que [2] le fait, soit des fonction bidimensionnelles $f(x, y)$, où x et y sont des coordonnées spatiales planes et où f associerait à chaque paire une amplitude ou valeur correspondante, ou plus simplement comme étant une carte ou grille de points de pixels, de largeur et de hauteur donné. Par définition les images matricielles présente une limite lié à l'agrandissement de l'image, Effectivement, plus l'image est agrandi plus la perte de détails s'accroît, la mise à l'échelle est donc impossible par la nature fixe du nombre de pixels constituant l'image.

1.2.1.1 Pixels

Un pixel, contraction de *picture element*, représente l'unité élémentaire d'une image numérique. Il correspond à un point de la grille matricielle et possède une position, définie par ses coordonnées spatiales, ainsi qu'une valeur décrivant son intensité ou sa couleur. Dans une image en niveaux de gris, cette valeur correspond généralement à une intensité lumineuse, tandis que dans une image couleur elle est souvent représentée par plusieurs composantes, par exemple rouge, vert et bleu. Ainsi, une image matricielle peut être comprise comme un ensemble fini de pixels organisés selon une largeur et une hauteur données [2]. Cette représentation rend le contenu directement dépendant de la résolution : lorsqu'une image est fortement agrandie, les pixels deviennent progressivement visibles, ce qui produit l'effet de pixellisation. Cette limite explique en partie l'intérêt de la vectorisation, dont l'objectif est de remplacer cette grille fixe de pixels par une représentation fondée sur des formes et des paramètres vectoriels [3].

1.2.2 Images vectorielles

Les images vectorielles sont décrites par un ensemble de formes géométriques individuelles : (cercles, segments de droites, courbes de Bézier, etc.), chacune de ces formes est enregistrée accompagnée de paramètres mathématiques et attributs la décrivant (couleur de remplissage, point d'encrage, etc.). Contrairement aux images représentées sous forme matricielles où les pixels sont figés, ici les images offrent une possibilité d'agrandissement théoriquement infini, et cela sans dégrader la qualité de l'image en question car le rendu image est calculé de nouveau pour chaque résolution [4]. Les images vectorielles sont donc très utilisées dans le graphisme et la conception, rendant donc le passage d'image matricielle à une image vectorielle d'autant plus critique pour ces domaines d'application, mais aussi complexe dû à la nature différente des représentations matricielle et vectorielle.

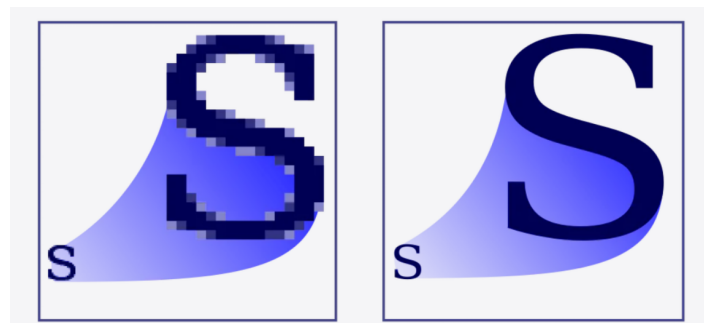


FIGURE 1.1 – Comparaison entre une image matricielle (gauche) sujette à la pixellisation lors de l'agrandissement, et une image vectorielle (droite) conservant sa netteté grâce à sa définition mathématique

1.2.2.1 Courbes de Bézier

Une courbe de Bézier est définie comme une fonction polynomiale dont la forme est dictée par la position de points de contrôle. La courbe passe par le premier et le dernier point (points d'ancrage), tandis que les points intermédiaires agissent comme des "aimants" qui tirent la courbe vers eux. Les courbes de Bézier font partie des formes géométriques individuelles qui composent les images vectorielles et se distinguent par le faible nombre de données numériques nécessaires à la représentation de formes complexes

1.2.2.2 Format SVG

SVG (Scalable Vector Graphics) est un format basé sur XML comportant des directives pour la définition de graphiques 2D[5]. Stockées dans des fichiers textes rendant les SVG compressible et indexable, ces directives SVG supportent aussi bien les formes géométriques vectorielle que les images et le texte. Pour les raisons précédentes ce sera ce format qui sera préféré pour la sortie du modèle.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://
www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg width="391" height="391" viewBox="-70.5 -70.5 391 391"
xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://
www.w3.org/1999/xlink">
<rect fill="#fff" stroke="#000" x="-70" y="-70" width="390"
height="390"/>
<g opacity="0.8">
  <rect x="25" y="25" width="200" height="200" fill="lime"
stroke-width="4" stroke="pink" />
  <circle cx="125" cy="125" r="75" fill="orange" />
  <polyline points="50,150 50,200 200,200 200,100" stroke="red"
stroke-width="4" fill="none" />
  <line x1="50" y1="50" x2="200" y2="200" stroke="blue" stroke-
width="4" />
</g>
</svg>
```

FIGURE 1.2 – Exemple du contenu d'un fichier .svg

1.3 Apprentissage automatique et apprentissage profond

1.3.1 Apprentissage automatique

L'apprentissage automatique est une discipline du domaine de l'intelligence artificielle, où contrairement à l'approche programmatique classique, la machine ne se voit pas présenté des instructions détaillé pour effectuer un ensemble de tâches T car les instructions nécessaires seraient trop complexes ou inefficaces[6], cette dernière démarre plutôt d'un ensemble de données à partir duquel elle pourra améliorer sa performance P

en se basant sur ses expériences passées E , en d'autres termes, converger vers une solution optimale en minimisant les erreurs commises lors des itérations précédentes [7]. Le type d'ensemble de données justement, distingue l'apprentissage automatique supervisé (où les données sont étiquetées) et non supervisé (où l'on cherche des structures cachées).

1.3.1.1 Apprentissage supervisé

L'apprentissage supervisé se caractérise par la présence des cibles ou réponses attendues t pour chaque variable d'entrée de l'ensemble de données d'entraînement. On parle de classification lorsque la variable t est discrète, et de régression lorsque celle-ci est continue [8]. Seulement cette approche est grandement limitée par la qualité des données étiquetées et leur disponibilité la rendant difficile pour certains cas d'utilisation.

1.3.1.2 Apprentissage non supervisé

L'apprentissage non supervisé intervient lorsque l'ensemble de données d'entraînement ne contient pas de cibles ou d'étiquettes associées aux variables d'entrée. Contrairement à l'apprentissage supervisé, le modèle doit donc découvrir par lui-même les structures présentes dans les données. Ces structures peuvent prendre la forme de groupes d'exemples similaires, comme dans le clustering, d'une estimation de la distribution des données, ou encore d'une projection vers un espace de plus faible dimension afin de faciliter la visualisation et l'analyse [8]. Cette approche est particulièrement utile lorsque l'annotation manuelle des données est coûteuse ou difficile, car elle permet d'exploiter directement de grands ensembles de données non étiquetées [6].

1.3.1.3 Apprentissage auto-supervisé

L'apprentissage auto-supervisé est une approche dans laquelle le modèle apprend à partir de données non étiquetées, mais en générant automatiquement des signaux de supervision à partir des données elles-mêmes. Contrairement à l'apprentissage supervisé classique, il ne nécessite donc pas d'annotations humaines explicites. Par exemple, dans le cas des images, une partie de l'image peut être masquée et le modèle est entraîné à reconstruire la partie manquante ; l'image originale joue alors le rôle de cible d'apprentissage [6]. Cette stratégie est particulièrement intéressante en vision par ordinateur, car elle permet d'exploiter de grands ensembles d'images non annotées afin d'apprendre des représentations visuelles générales, qui peuvent ensuite être réutilisées ou adaptées à d'autres tâches comme la classification, la détection ou la segmentation [9].

1.3.2 Apprentissage profond

L'apprentissage profond est une sous-branche de l'apprentissage automatique fondée principalement sur les réseaux de neurones profonds. Ces modèles sont constitués de plusieurs couches de neurones artificiels capables d'apprendre progressivement des représentations de plus en plus abstraites à partir des données. Cette approche s'est fortement développée grâce à l'augmentation de la puissance de calcul, à la disponibilité de grandes quantités de données et aux progrès réalisés dans l'entraînement des réseaux profonds [6]. Dans le domaine du traitement d'images, l'apprentissage profond est particulièrement important, notamment à travers les réseaux de neurones convolutionnels, capables de prendre directement des images en entrée et d'apprendre automatiquement les caractéristiques utiles pour des tâches complexes de classification, de reconnaissance ou de reconstruction [2].

1.3.2.1 Réseaux de neurones

Les réseaux de neurones artificiels sont des modèles d'apprentissage automatique inspirés, de manière simplifiée, du fonctionnement des neurones biologiques. Ils sont constitués d'unités de calcul appelées neurones artificiels, organisées en couches et reliées entre elles par des connexions pondérées. L'apprentissage consiste à ajuster ces poids à partir des données d'entraînement afin que le réseau puisse produire une sortie correcte pour une tâche donnée [6]. Dans le domaine du traitement d'images, ces modèles sont utilisés pour apprendre automatiquement des fonctions de décision à partir d'exemples, notamment dans les problèmes de classification, de reconnaissance de formes et de reconstruction [2].

1.3.2.2 Réseaux convolutionnels

Les réseaux de neurones convolutionnels, ou CNN, sont une catégorie de réseaux de neurones profonds particulièrement adaptée au traitement des images. Contrairement aux réseaux entièrement connectés, ils exploitent directement la structure spatiale de l'image grâce à des opérations de convolution appliquées sur des régions locales appelées champs récepteurs. Ces convolutions utilisent des noyaux dont les poids sont appris durant l'entraînement afin de produire des cartes de caractéristiques capables de détecter des motifs visuels comme les contours, les formes ou certaines textures [2]. Les couches de pooling sont ensuite utilisées pour réduire la dimension des cartes de caractéristiques, diminuer le coût de calcul et limiter le risque de surapprentissage, tout en conservant les informations les plus importantes [6].

1.3.2.3 Transformers

Les Transformers sont une architecture de réseaux de neurones introduite initialement pour le traitement des séquences, notamment en traduction automatique. Contrairement aux réseaux récurrents, ils ne traitent pas les éléments de la séquence de manière strictement séquentielle, mais s'appuient principalement sur des mécanismes d'attention, en particulier l'auto-attention, afin de modéliser les relations entre les différents éléments d'une entrée [10]. Cette architecture a ensuite été largement utilisée dans de nombreux domaines de l'apprentissage profond, notamment le traitement du langage naturel, la vision par ordinateur et les modèles multimodaux, grâce à sa capacité à apprendre des dépendances complexes et à être entraînée efficacement sur de grandes quantités de données [6].

1.3.2.4 Vision Transformers

Les Vision Transformers, ou ViT, sont une adaptation de l'architecture Transformer au traitement des images. Au lieu de s'appuyer principalement sur des convolutions comme les CNN, une image est divisée en petits blocs appelés patches, par exemple de taille 16×16 . Ces patches sont ensuite transformés en vecteurs et traités comme une séquence d'éléments, de manière similaire aux tokens dans le traitement du langage naturel [11]. Grâce au mécanisme d'auto-attention, le modèle peut apprendre des relations globales entre différentes régions de l'image. Dans le contexte de la vectorisation d'images, cette approche peut être utilisée pour extraire des caractéristiques visuelles avant de prédire des paramètres vectoriels ; par exemple, SuperSVG emploie un encodeur ViT pour encoder les superpixels avant de générer les chemins SVG correspondants [3].

1.4 Mécanismes d'attention

Les mécanismes d'attention constituent une composante importante des architectures modernes d'apprentissage profond, en particulier dans les modèles de type Transformer. Leur objectif est de permettre au modèle d'accorder une importance différente aux éléments d'une entrée selon leur pertinence pour la tâche à résoudre. Au lieu de traiter toutes les informations de manière uniforme, l'attention permet de pondérer certaines parties de l'entrée afin de mieux capturer les relations entre les éléments. Cette idée est particulièrement utile pour modéliser des dépendances à longue distance, que ce soit dans une séquence de mots ou dans une image découpée en régions ou en patches [10]. Dans les Transformers, ces mécanismes remplacent les traitements séquentiels classiques et permettent un apprentissage plus parallélisable [10].

1.4.1 Attention

Le mécanisme d'attention permet à un modèle de sélectionner dynamiquement les informations les plus pertinentes dans une entrée. Son fonctionnement peut être décrit à l'aide de trois éléments principaux : une requête, des clés et des valeurs. La requête représente l'information recherchée, les clés servent à mesurer la similarité avec cette requête, et les valeurs contiennent les informations à combiner. Le modèle calcule alors des scores de similarité entre la requête et les différentes clés, puis applique généralement une fonction softmax afin d'obtenir des poids d'attention. La sortie est ensuite obtenue comme une somme pondérée des valeurs, ce qui permet au modèle de se concentrer davantage sur les éléments les plus utiles [10]. De manière intuitive, l'attention peut être vue comme un mécanisme de récupération d'information différentiable, où le modèle apprend à retrouver les éléments importants d'une séquence ou d'une représentation en fonction du contexte [6].

1.4.2 Self-attention

La self-attention, ou auto-attention, est un cas particulier du mécanisme d'attention dans lequel les requêtes, les clés et les valeurs proviennent de la même séquence. Elle permet donc à chaque élément d'une entrée de prendre en compte les autres éléments de cette même entrée afin de construire une représentation plus riche et contextualisée. Dans une phrase, par exemple, chaque mot peut être relié aux autres mots pour mieux comprendre son sens selon le contexte. Dans une image représentée sous forme de patches, chaque patch peut de la même manière interagir avec les autres régions de l'image [10]. Cette capacité à modéliser les relations internes d'une séquence explique l'importance de la self-attention dans les architectures Transformer. Dans le cas de SuperSVG, un module de self-attention est utilisé après le module de cross-attention afin de traiter les caractéristiques intermédiaires et de les projeter vers l'espace des paramètres des chemins SVG [3].

1.4.3 Cross-attention

La cross-attention, ou attention croisée, est une variante du mécanisme d'attention dans laquelle les requêtes ne proviennent pas de la même source que les clés et les valeurs. Contrairement à la self-attention, où un élément interagit avec les autres éléments de la même séquence, la cross-attention permet à une représentation de s'appuyer sur une autre représentation externe. Dans un Transformer encodeur-décodeur, par exemple, le décodeur produit les requêtes, tandis que les clés et les valeurs proviennent des représentations calculées par l'encodeur. Cela permet au décodeur de sélectionner les informations importantes de l'entrée lors de la génération de la sortie [6]. Dans le

contexte de la vectorisation d'images, SuperSVG utilise un module de cross-attention pour calculer la corrélation entre les caractéristiques extraites d'un superpixel par un encodeur ViT et des requêtes apprenables associées aux chemins SVG. Ce mécanisme permet ainsi de produire des caractéristiques intermédiaires adaptées à la prédiction des paramètres vectoriels [3].

1.5 Concepts utilisés dans la vectorisation par apprentissage profond

La vectorisation d'images par apprentissage profond repose sur plusieurs concepts techniques permettant de passer d'une image matricielle, composée de pixels, à une représentation vectorielle décrite par des formes paramétriques. Contrairement aux méthodes classiques de traçage, les approches récentes cherchent à apprendre ou à optimiser automatiquement les paramètres des primitives vectorielles afin de reconstruire au mieux l'image d'entrée. Pour cela, elles utilisent notamment la décomposition de l'image en régions plus simples, le rendu différentiable pour relier les paramètres vectoriels à leur rendu matriciel, ainsi que des fonctions de perte permettant de mesurer l'écart entre l'image originale et l'image reconstruite. Certaines méthodes adoptent également une stratégie progressive de type *coarse-to-fine*, dans laquelle la structure globale de l'image est d'abord reconstruite avant d'ajouter les détails fins [12, 3].

1.5.1 Superpixels

Les superpixels sont des régions obtenues en regroupant des pixels voisins présentant des caractéristiques similaires, notamment en termes de couleur et de position spatiale. Ils permettent de remplacer une représentation pixel par pixel par une représentation plus compacte, composée de régions visuellement cohérentes. Parmi les méthodes les plus connues, l'algorithme SLIC (*Simple Linear Iterative Clustering*) adapte le principe du k-means afin de générer efficacement des superpixels. Cette méthode est appréciée pour sa simplicité, sa rapidité, sa faible consommation mémoire et sa capacité à produire des régions qui respectent bien les contours de l'image [13]. Dans le cadre de la vectorisation d'images, les superpixels sont utiles car ils divisent une image complexe en sous-régions plus simples à traiter. SuperSVG exploite précisément cette idée en décomposant l'image d'entrée en superpixels contenant des pixels de couleurs et de textures similaires, puis en vectorisant ces régions de manière progressive [3].

1.5.2 Rendu différentiable

Le rendu différentiable est une technique qui permet de relier une représentation vectorielle à son rendu matriciel tout en conservant la possibilité de calculer des gradients. Dans le contexte de la vectorisation d'images, les paramètres des primitives vectorielles, comme les positions des points de contrôle, les couleurs ou l'épaisseur des traits, sont d'abord rendus sous forme d'image raster. L'image obtenue est ensuite comparée à l'image cible à l'aide d'une fonction de perte, puis les gradients de cette perte sont rétropropagés vers les paramètres vectoriels afin de les optimiser [12]. Cette approche permet donc d'utiliser des méthodes d'optimisation ou d'apprentissage profond pour ajuster automatiquement les formes vectorielles à partir d'une supervision dans le domaine des pixels. Elle est notamment utilisée dans des méthodes comme LIVE, qui optimise progressivement des chemins de Bézier dans un cadre de rendu entièrement différentiable afin de produire des SVG organisés en couches [14].

1.5.3 Fonctions de perte

Les fonctions de perte jouent un rôle central dans la vectorisation d'images par apprentissage profond, car elles permettent de mesurer l'écart entre l'image matricielle d'origine et l'image reconstruite à partir des primitives vectorielles. Dans les approches utilisant un rendu différentiable, les chemins SVG prédits ou optimisés sont d'abord rasterisés, puis comparés à l'image cible à l'aide d'une fonction de perte calculée dans l'espace des pixels. Une perte simple, comme la MSE, peut être utilisée pour minimiser la différence entre les deux images, mais elle ne suffit pas toujours à préserver correctement la structure, les contours ou la couleur des objets. Par exemple, LIVE montre qu'une optimisation fondée uniquement sur la MSE peut conduire à un effet de couleur moyenne, puis propose une perte UDF afin de donner plus d'importance aux pixels proches des contours, ainsi qu'une perte Xing pour limiter les auto-intersections des courbes [14]. Dans SuperSVG, les auteurs combinent plusieurs pertes adaptées à la vectorisation par superpixels, notamment une perte de reconstruction normalisée, une perte de frontière pour empêcher les chemins de dépasser les limites du superpixel, et une perte Dynamic Path Warping permettant au modèle de raffinement d'hériter des connaissances du modèle grossier [3].

1.5.4 Apprentissage coarse-to-fine

L'apprentissage *coarse-to-fine* désigne une stratégie progressive dans laquelle le modèle commence par reconstruire les éléments globaux ou principaux d'une image, avant d'ajouter progressivement des détails plus fins. Dans le contexte de la vectorisation d'images, cette approche est particulièrement utile car une image complexe est difficile

à convertir directement en une représentation SVG précise. En procédant par étapes, le modèle peut d'abord approximer la structure générale, puis améliorer localement la reconstruction. SuperSVG adopte cette logique à travers un cadre d'apprentissage en deux étapes : un modèle grossier prédit d'abord des chemins SVG pour reconstruire la structure principale du superpixel, puis un modèle de raffinement ajoute d'autres chemins afin d'enrichir les détails, en étant guidé par les chemins prédits à l'étape précédente [3]. Une idée similaire apparaît dans LIVE, où l'image est reconstruite progressivement en ajoutant et en optimisant de nouveaux chemins de Bézier dans une représentation en couches, ce qui permet d'obtenir un SVG plus compact et plus cohérent avec la structure visuelle de l'image [14].

1.6 Évaluation des résultats

L'évaluation des résultats est une étape essentielle pour mesurer la qualité d'une méthode de vectorisation d'images. Dans ce contexte, l'image SVG générée est généralement rendue sous forme matricielle, puis comparée à l'image d'entrée afin d'estimer la fidélité de reconstruction. Plusieurs métriques complémentaires peuvent être utilisées, car une seule mesure ne suffit pas toujours à juger correctement la qualité visuelle d'un résultat. Les métriques comme la MSE et le PSNR évaluent principalement l'écart pixel par pixel entre l'image originale et l'image reconstruite, tandis que la SSIM cherche davantage à mesurer la similarité structurelle. De son côté, la LPIPS évalue une distance perceptuelle plus proche de la perception humaine. Enfin, le temps d'inférence permet d'évaluer l'efficacité pratique de la méthode, notamment lorsqu'il s'agit de traiter des images complexes ou un grand nombre d'images [3].

1.6.1 MSE

La MSE (*Mean Squared Error*), ou erreur quadratique moyenne, est une métrique utilisée pour mesurer l'écart pixel par pixel entre une image de référence et une image reconstruite. Dans le cas de la vectorisation, l'image SVG générée est d'abord rendue sous forme matricielle, puis comparée à l'image d'entrée. La MSE calcule la moyenne des carrés des différences entre les valeurs des pixels des deux images :

$$MSE = \frac{1}{HWC} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^C (I_{i,j,c} - \hat{I}_{i,j,c})^2$$

où I représente l'image originale, $\hat{I}$ l'image reconstruite, H et W les dimensions de l'image, et C le nombre de canaux. Une valeur faible de MSE indique donc une meilleure fidélité de reconstruction. Cette métrique est utilisée dans SuperSVG pour mesurer la distance pixel entre l'image d'entrée et le rendu obtenu à partir du SVG généré [3].

Cependant, la MSE reste une mesure purement numérique et ne reflète pas toujours parfaitement la qualité visuelle perçue. Dans LIVE, les auteurs montrent par exemple qu'une optimisation basée uniquement sur la MSE peut conduire à un effet de couleur moyenne, car tous les pixels sont pris en compte de la même manière, même ceux qui ne sont pas directement liés à la forme en cours d'optimisation [14].

1.6.2 PSNR

Le PSNR (*Peak Signal-to-Noise Ratio*) est une métrique dérivée de la MSE qui permet d'évaluer la qualité de reconstruction d'une image. Il mesure le rapport entre la valeur maximale possible d'un pixel et l'erreur introduite par la reconstruction. Dans le cas d'images normalisées entre 0 et 1, il peut être exprimé par :

$$PSNR = 10 \log_{10} \left(\frac{1}{MSE} \right)$$

De manière générale, une valeur élevée de PSNR indique que l'image reconstruite est proche de l'image originale, tandis qu'une valeur faible traduit une erreur de reconstruction plus importante. Dans le contexte de la vectorisation, le PSNR est calculé après avoir rendu l'image SVG sous forme matricielle, puis en la comparant à l'image d'entrée. SuperSVG utilise cette métrique avec la MSE afin de mesurer la distance pixel entre l'image d'entrée et le rendu SVG obtenu [3]. De même, DiffVG emploie le PSNR pour évaluer l'amélioration de la fidélité visuelle après optimisation des paramètres vectoriels, ce qui confirme son intérêt pour comparer quantitativement les résultats de vectorisation [12].

1.6.3 SSIM

La SSIM (*Structural Similarity Index Measure*) est une métrique d'évaluation de qualité d'image qui mesure la similarité structurelle entre une image de référence et une image reconstruite. Contrairement à la MSE et au PSNR, qui se concentrent principalement sur les différences pixel par pixel, la SSIM cherche à comparer des informations plus proches de la perception visuelle humaine, notamment la luminance, le contraste et la structure locale de l'image [15]. Elle peut être exprimée sous la forme :

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

où x et y représentent deux fenêtres locales des images comparées, μ_x et μ_y leurs moyennes, σ_x^2 et σ_y^2 leurs variances, σ_{xy} leur covariance, et C_1 , C_2 des constantes de stabilisation. Une valeur élevée de SSIM indique une meilleure préservation de la structure visuelle. Dans le contexte de la vectorisation, SuperSVG utilise la SSIM pour

mesurer la distance structurelle entre l'image d'entrée et le rendu obtenu à partir du SVG généré [3].

1.6.4 LPIPS

La LPIPS (*Learned Perceptual Image Patch Similarity*) est une métrique perceptuelle qui mesure la similarité entre deux images à partir de caractéristiques extraites par des réseaux de neurones profonds. Contrairement à la MSE et au PSNR, qui comparent directement les valeurs des pixels, la LPIPS compare les représentations internes obtenues à différents niveaux d'un réseau pré-entraîné. Cette approche permet de mieux prendre en compte la perception visuelle humaine, car deux images peuvent avoir une faible différence pixel par pixel tout en étant visuellement différentes, ou inversement [16]. De manière simplifiée, la LPIPS calcule une distance entre les activations normalisées de deux images :

$$LPIPS(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\phi_l(x)_{h,w} - \phi_l(\hat{x})_{h,w})\|_2^2$$

où ϕ_l représente les caractéristiques extraites à la couche l , w_l des poids appris, et $\hat{x}$ l'image reconstruite. Une valeur faible de LPIPS indique donc une meilleure similarité perceptuelle. Dans le contexte de la vectorisation, SuperSVG utilise la LPIPS pour évaluer la distance perceptuelle entre l'image d'entrée et le rendu obtenu à partir du SVG généré [3].

1.6.5 Temps d'inférence

Le temps d'inférence désigne le temps nécessaire à un modèle pour produire un résultat à partir d'une image d'entrée. Dans le contexte de la vectorisation d'images, il correspond au temps requis pour générer les chemins SVG ou pour optimiser les paramètres vectoriels représentant l'image. Cette métrique est importante car une méthode peut obtenir une bonne qualité de reconstruction tout en restant difficilement exploitable si son temps de calcul est trop élevé. Les méthodes fondées sur l'optimisation directe des paramètres SVG peuvent produire des résultats précis, mais elles nécessitent souvent plusieurs itérations de rendu et de rétropropagation, ce qui augmente fortement le temps de traitement. À l'inverse, les méthodes fondées sur l'apprentissage profond cherchent à prédire directement les paramètres vectoriels, ce qui permet généralement une génération plus rapide. Dans SuperSVG, les auteurs évaluent ainsi le temps d'exécution en plus des métriques de qualité comme la MSE, le PSNR, la LPIPS et la SSIM. Leurs résultats montrent que SuperSVG-B atteint un temps d'inférence très court, par exemple 0,31 seconde pour 500 chemins et 0,71 seconde pour 2000 chemins, tandis que certaines méthodes d'optimisation comme LIVE nécessitent un temps beaucoup plus important

[3].

1.7 Travaux connexes

1.7.1 Approches algorithmiques

1.7.1.1 Potrace

1.7.2 Approches basées sur l'optimisation

1.7.2.1 DiffVG

1.7.2.2 LIVE

1.7.3 Approches basées sur l'apprentissage profond

1.7.3.1 Im2Vec

1.7.3.2 SuperSVG

1.8 Synthèse comparative des approches

1.9 Problématique

1.10 Conclusion

Chapitre 2

Approche Proposée

2.1 Introduction

2.2 Dataset et prétraitement

2.2.1 Le prétraitement des données

2.2.1.1 Former les paires d'images

2.2.1.2 Augmentation de données

2.3 Méthode proposée

2.3.1 Définir les hyperparamètres évalués par les métaheuristiques

2.3.1.1 L'Algorithme Génétique

2.3.1.2 Les paramètres à optimiser

2.3.1.3 Évaluation d'une solution

2.3.2 Architecture

2.3.2.1 Sur-échantillonnage (Upsampling)

2.3.2.2 Sous-échantillonnage (Downsampling)

2.4 Conclusion

Chapitre 3

Expérimentations et Résultats

3.1 Introduction

3.2 Environnement de développement

3.2.1 Hardware

TABLE 3.1 – Spécifications de la machine Kaggle

CPU	Mémoire	GPU
1,5 GHz	13 Go RAM + 73,1 Go HDD	Nvidia P100 — 16 Go VRAM

3.2.2 Software

3.3 Dataset utilisé

3.4 Prétraitement

3.5 Évaluation

3.5.1 Évaluation quantitative

3.5.2 Fonction de perte

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.1)$$

3.6 Paramétrage de l’Algorithme Génétique

- Taille de population : 10
- Nombre de générations : 6
- Probabilité de mutation : 0,5
- Fonction objective : PSNR

3.7 Sous-échantillonnage

3.7.1 Hyperparamètres avec la métaheuristique (GA)

TABLE 3.2 – Valeurs du PSNR en fonction du nombre de générations

Générations	PSNR (dB)
3	50,079
6	50,219

- Batch size : 19 Kernel size : 3×3
- Epochs : 27 Nombre de filtres : 64
- Nombre de couches : 5
- Learning rate : Adam Optimizer ($10^{-4} \rightarrow 10^{-5}$, limite 5000)

3.7.2 Résultats

3.7.2.1 Résultats quantitatifs

TABLE 3.3 – Résultats du downsampling (Set5) — PSNR / SSIM

Dataset	Facteur	Notre Modèle	Bicubique
Set5	$\times 2$	45,61 / 0,99	36,17 / 0,97
	$\times 3$	44,80 / 0,99	30,35 / 0,94
	$\times 4$	33,85 / 0,97	26,80 / 0,91

3.7.2.2 Résultats qualitatifs

3.8 Sur-échantillonnage

3.8.1 Hyperparamètres

- Batch size : 16 Kernel size : 3×3
- Epochs : 250 Nombre de filtres : 128
- Nombre de blocs résiduels : 12
- Learning rate : Adam Optimizer ($10^{-4} \rightarrow 10^{-5}$, limite 5000)

3.8.2 Résultats

3.8.2.1 Résultats quantitatifs

TABLE 3.4 – Comparaison quantitative sur Set5 (PSNR en dB)

Test set	Facteur	Bicubic	SRGAN	DRDN	EDSAN	CAR	SRCNN	Notre Modèle
Set5	×2	33,66	–	32,95	35,25	34,38	36,66	36,71
	×3	30,39	–	29,18	31,48	–	32,75	32,20
	×4	28,42	29,40	27,35	29,80	28,42	30,49	31,498

3.8.2.2 Résultats qualitatifs

3.9 Conclusion

Conclusion Générale

Au cours de ce projet de fin d'étude, nous avons exploré divers aspects de l'apprentissage profond et de la vision par ordinateur pour résoudre le problème du redimensionnement des images.

Bibliographie

- [1] Vasant DHAR. “The Paradigm Shifts in Artificial Intelligence”. In : *Communications of the ACM* 67.11 (nov. 2024), p. 50-59. DOI : 10.1145/3664804.
- [2] Rafael C. GONZALEZ et Richard E. WOODS. *Digital Image Processing*. 4th. Pearson, 2018. ISBN : 978-1-292-22304-9.
- [3] Teng HU et al. “SuperSVG : Superpixel-based Scalable Vector Graphics Synthesis”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, p. 24892-24902.
- [4] James D. FOLEY et al. *Computer Graphics : Principles and Practice*. 2nd. Addison-Wesley Professional, 1996, p. 8.
- [5] W3C. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. World Wide Web Consortium, 2011. URL : <https://www.w3.org/TR/SVG11/>.
- [6] Aurélien GÉRON. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd. O'Reilly Media, 2022. ISBN : 978-1-098-12597-4.
- [7] Tom M. MITCHELL. *Machine Learning*. McGraw-Hill, 1997, p. 2.
- [8] Christopher M. BISHOP. *Pattern Recognition and Machine Learning*. Springer, 2006, p. 3.
- [9] Longlong JING et Yingli TIAN. “Self-Supervised Visual Feature Learning with Deep Neural Networks : A Survey”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11 (2021), p. 4037-4058. DOI : 10.1109/TPAMI.2020.2992393.
- [10] Ashish VASWANI et al. “Attention Is All You Need”. In : *Advances in Neural Information Processing Systems*. T. 30. Curran Associates, Inc., 2017.
- [11] Alexey DOSOVITSKIY et al. “An Image Is Worth 16x16 Words : Transformers for Image Recognition at Scale”. In : *International Conference on Learning Representations*. 2021. URL : <https://arxiv.org/abs/2010.11929>.
- [12] Tzu-Mao LI et al. “Differentiable Vector Graphics Rasterization for Editing and Learning”. In : *ACM Transactions on Graphics* 39.6 (2020), 193 :1-193 :15. DOI : 10.1145/3414685.3417871.
- [13] Radhakrishna ACHANTA et al. “SLIC Superpixels Compared to State-of-the-Art Superpixel Methods”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.11 (2012), p. 2274-2282. DOI : 10.1109/TPAMI.2012.120.

- [14] Xu MA et al. “Towards Layer-wise Image Vectorization”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, p. 16314-16323. DOI : 10.1109/CVPR52688.2022.01583.
- [15] Zhou WANG et al. “Image Quality Assessment : From Error Visibility to Structural Similarity”. In : *IEEE Transactions on Image Processing* 13.4 (2004), p. 600-612. DOI : 10.1109/TIP.2003.819861.
- [16] Richard ZHANG et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, p. 586-595. DOI : 10.1109/CVPR.2018.00068.